

tmux documentation

basic-digital tmux — Project Constitution

Field	Value
Revision	3
Created	2026-05-07
Last modified	2026-05-21
Status	active
Extends	<code>constitution/Constitution.md</code> — Helix Universal Constitution, pinned 7f738df
Status summary	R3 (2026-05-21): full refactor to the HelixConstitution extends-template form. Universal clauses now inherited from the <code>constitution/</code> submodule; this file holds only Project Articles §101–§109. No universal clause weakened — inheritance is verified by <code>scripts/tests/test_constitution_inheritance.sh</code> + its paired mutation.

This constitution **extends** the Helix Universal Constitution at [constitution/Constitution.md](#). Every clause there — the anti-bluff covenant (§11.4), absolute data safety (§9), the 60 % host-memory budget, the continuation-document invariant (§12.10), the no-guessing mandate (§11.4.6), commit/push discipline, credentials handling — applies to this repository **unconditionally**.

The Project Articles below **extend** those universal clauses with tmux-specific rules. They never weaken or override a universal clause. **When this file disagrees with the constitution submodule, the constitution wins.**

Inheritance is verified, not assumed. `scripts/tests/test_constitution_inheritance.sh` asserts the submodule is present at the pinned revision and that the §11.4 *End-user Quality Guarantee* anchor physically exists in `constitution/Constitution.md`. Its paired mutation — `constitution/meta_test_inheritance.sh`, wired as the CM-CONSTITUTION-INHERITANCE entry in `scripts/tests/meta_test_false_positive_proof.sh` — deletes that anchor and proves the gate then FAILs. The gate cannot itself be a bluff gate.

Project Articles

§101 — Anti-bluff covenant binding (origin anchor)

This repository is one of the historical origins of the Helix anti-bluff covenant. The verbatim user mandate that the universal §11.4 codifies is preserved here for forensic continuity — it **MUST** remain in this project's Constitution regardless of inheritance:

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This

MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

The bar for shipping is "users can use the feature," not "tests pass." Every PASS in this codebase — unit test OR HelixQA Challenge — MUST carry positive runtime evidence captured *during execution* that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-only PASS are critical defects regardless of how green the summary line is. **FAIL-bluffs** (a test that exits FAIL for a script-internal reason, not a product defect) are equally forbidden and MUST be fixed at the source layer.

The full covenant — recorded-evidence requirement, FAIL-bluff prohibition, no-guessing mandate, test-interrupt-on-discovery — is universal §11.4.1 through §11.4.6. This article binds this project to it and adds the project-specific articles below.

§102 — Operator-path test coverage (project rule, User mandate 2026-05-13)

Forensic anchor: Fixed.md A12 + A13. Tests 11 and 14 reported GREEN while their operator-facing equivalents (`tmx new -s X`) were broken — test 11 only exercised the explicit-socket path; test 14 hand-spawned `systemd-run --user --scope` units while the real `tmx new` workflow placed all sessions in one shared cgroup.

The mandate. Every gate test for a feature MUST exercise the SAME entry point an end-user would invoke in production. Tests that bypass the operator's wrapper / helper / install path and reproduce its effects with hand-crafted equivalents DO NOT satisfy the universal §11.4.2 captured-evidence requirement. When the operator's path and the test's path diverge:

1. The test header MUST EXPLICITLY name the divergence (e.g. "test invokes `tmux` directly; operator path is `tmx new`").
2. A SEPARATE end-to-end test MUST close that divergence with captured evidence on the operator-facing entry point.

Operative test: for every test under `scripts/tests/`, ask "would an operator hit this code path in their normal workflow?" If no, that test is supplementary; the operator-path test must exist alongside it.

No grep-on-script-content alone. A `grep` on a script for a literal string is allowed AS A STATIC CHECK *in addition to* a runtime readback — never as the only assertion.

Layer-4 mutations MUST target the operator-path code. Mutations target `scripts/tmx-vm/scripts/tmx.template/scripts/tmux.conf.template` (the code that runs in production), not synthetic-test code.

Non-compliance is a release blocker.

§103 — Four-layer test coverage for `tmux` defects (User mandate 2026-05-06)

Every defect found, fixed, or introduced in this repo MUST land all four layers before the cycle closes:

- **Layer 1 — pre-build / static-source gate.** A check in `scripts/verify.sh` (or the relevant build script) that catches the defect class at source.
- **Layer 2 — runtime test.** A numbered test in `scripts/tests/NN_*.sh` producing positive runtime artifacts, anti-bluff per §101, operator-path per §102, topology-dispatched per §104.

- **Layer 3 — HelixQA Challenge.** An entry in `scripts/challenges/tmux.yaml` carrying captured-evidence semantics. A test without a paired Challenge is a §101 PASS-bluff.
- **Layer 4 — paired mutation.** A mutation in `scripts/tests/meta_test_false_positive_proof.sh` proving the Layer-2 test catches its own negation (mutation CAUGHT) and that the feature is restored afterwards (RESTORED).

Test-interrupt-on-discovery. The moment any defect is rediscovered, re-produced, or newly identified during a test cycle, the cycle STOPS. Fix at root cause → land all four layers → full rebuild → repeat the full cycle from the beginning.

§104 — Per-host-topology test dispatch for tmux (User mandate 2026-05-05)

Tests that depend on host topology (systemd version, cgroup v1 vs v2, distro-specific controller availability, presence of `systemd-run --user`, Linux vs Darwin) MUST detect topology at test entry and dispatch the topology-appropriate variant — recording the probe output as positive evidence of the SKIP rationale. A test running the wrong variant for the actual topology and PASSing is a §101 PASS-bluff. Silent degradation to a no-isolation path is forbidden.

`scripts/tmx` classifies the host as `tmx-supported` / `tmx-degraded` / `tmx-unsupported` via `_probe_topology()` — the canonical dispatch seam.

§105 — Per-session isolation architecture (default since 2026-05-13)

Each `tmx new -s NAME` spawns its OWN tmux server on socket `tmx-NAME` inside its OWN OS-native isolation boundary:

- **Linux** — cgroup-v2 transient scope `tmx-NAME.scope` via `systemd-run --user --scope`. `MemoryMax` host-adaptive ($\max(\text{MemTotal} \times 60\% / 4, 2\text{ GB})$ unless `TMX_MEM` overrides), `CPUQuota=200%`, `TasksMax=4096`, `Delegate=yes`. OOM in any one scope affects ONLY that scope — `user.slice` survives.
- **macOS (Darwin)** — POSIX `rlimit` wrapper as session `default-command`: `RLIMIT_CPU + RLIMIT_NPROC`, kernel-enforced. `RLIMIT_AS` (memory) is NOT enforced by XNU for unprivileged processes — documented honestly per §101, see `docs/guide/README.md §5.6`.

The session shell is the operator's host shell with full `$PATH`. Plain-vanilla tmux UX with OS-native isolation as a safeguard.

§106 — TMX_MEM memory-budget enforcement (project enforcement of universal §12.6)

The universal 60 % host-memory budget is enforced in this project by the per-session containerization in `scripts/tmx`. Invariant: $\sum(\text{active TMX_MEM}) \leq 0.6 \times \text{MemTotal}$. Operator default `TMX_MEM=8G`. On Linux the cgroup `MemoryMax` is kernel-enforced; on macOS the cap is informational only (XNU gap, §105). There is NO operator-facing override of the per-host total.

§107 — Upstream tmux submodule pinned + immutable

`tmux/` references `tmux/tmux` at tag `3.6a` (commit `cc117b5`). It is a third-party upstream submodule: **never modified**, never advanced off the tag without an explicit, documented decision

in `Fixed.md` + a governance-doc update in the same commit. Silent pin-drift is a §101 / universal §11.4.6 violation (forensic precedent: `Fixed.md` §3.8).

§108 — Native dual-OS build discipline

The hardened `tmux` binary is built natively per OS — Linux ELF and macOS Mach-O — from the same `tmux/` submodule. `scripts/setup.sh`, `scripts/install_deps.sh`, and every project script MUST detect host OS and apply the correct action out of the box (no manual per-OS flags). Compile hardening, jemalloc linkage, and the verification gate apply to both targets.

§109 — Anti-bluff verification gate (load-bearing)

`scripts/verify.sh` is the single decision point for whether the binary is operator-safe. GREEN → `setup.sh` proceeds to PATH export; RED → `setup.sh` REFUSES to install. There is no "force install" mode and no flag that bypasses the gate.

Overrides of the Universal Constitution

None. This project introduces **no override** of any universal clause. Any future override MUST appear here under an explicit `### Override §X.Y — <reason>` heading with justification.

Owned-submodule set

Submodules this project owns and is responsible for:

Containers	<code>git@github.com:vasic-digital/Containers.git</code>	(cgroup orche
constitution	<code>git@github.com:HelixDevelopment/HelixConstitution.git</code>	(go

`tmux/` is a **third-party upstream** submodule (`tmux/tmux` tag 3.6a) — excluded from the owned set; immutable per §107.

Project-specific remotes

Repo	Remotes	Push mechanism
Main — <code>vasic-digital/tmux</code>	<code>github + gitlab</code>	<code>bash commit_all.sh "<msg>"</code> — ONLY allowed mechanism
Containers	<code>github + gitlab</code>	the Containers repo's own commit wrapper

Never `git push` directly on the main repo. SSH remotes only — no HTTPS for Git. No CI/CD pipeline files. Force-push is never automatic and requires explicit per-operation authorization per universal §9.

This Project Constitution governs the `vasic-digital/tmux` repository. It is subordinate to and extends `constitution/Constitution.md`.